LeetCode Practice

11 Medium: BFS

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

11
MEDIUM

Patterns (in order)

1. BFS (11)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

11 questions | tap a title to jump to it

Medium questions (11) Go >

BFS (11) Go >

M 1. Perfect Squares LC 279
M 2. Walls and Gates LC 286
M 3. Coin Change LC 322
M 4. Minimum Genetic Mutation LC 433
M 5. 01 Matrix LC 542
M 6. Open the Lock LC 752
M 7. Snakes and Ladders LC 909
M 8. Rotting Oranges LC 994
M 9. Shortest Path in Binary Matrix LC 1091
M 10. Minimum Knight Moves LC 1197
M 11. Shortest Path to Get Food LC 1730

Medium questions (11)

MEDIUM | TOPIC 1 OF 1

BFS

11 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Perfect Squares LC 279
2. Walls and Gates LC 286
3. Coin Change LC 322
4. Minimum Genetic Mutation LC 433
5. 01 Matrix LC 542
6. Open the Lock LC 752
7. Snakes and Ladders LC 909
8. Rotting Oranges LC 994
9. Shortest Path in Binary Matrix LC 1091
10. Minimum Knight Moves LC 1197
11. Shortest Path to Get Food LC 1730

MEDIUM | BFS | LeetCode #279 | 1 of 11 | 1/11

Perfect Squares

QUESTION

Given an integer n, return the least number of perfect square numbers that sum to n.

A perfect square is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares while 3 and 11 are not.

Example 1	
Input	`n = 12`
Output	**3**
Explanation	*12 = 4 + 4 + 4.*

Example 2	
Input	`n = 13`
Output	**2**
Explanation	*13 = 4 + 9.*

CONSTRAINTS

- $1 <= n <= 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the minimum number of perfect squares (1,4,9,...) that sum to n. Right?"

"A few quick questions:

Unlimited use of each perfect square? Yes (unbounded knapsack). n>=1?"

"Let me walk: n=12 -> 4+4+4=3 squares ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def fewest(n):
    if n == 0:
        return 0
    return 1 + the smallest fewest(n - s) over
        every perfect square s <= n
```

TIME	SPACE
O((sqrt n)^n)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Up to sqrt(n) branches at every level, and the same remainders repeat. A table fewest[i] = 1 + min fewest[i - s^2] is O(n * sqrt(n)); Lagrange's four-square theorem gives an O(sqrt n) answer.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"BFS on numbers, treating perfect squares as edges — O(n * sqrt(n)). Walk-through:"*

Pre-compute squares: squares = [i² for i <= sqrt(n)].

BFS from n: each step subtracts a square -> finds minimum steps (=min squares) to reach 0.

`queue = deque([n]). visited = {n}. steps = 0.`

While queue:

steps++. For _ in range(len(queue)):

`curr = queue.popleft().`

For sq in squares:

`nxt = curr - sq.`

If nxt == 0: return steps.

If nxt > 0 and nxt not in visited: visited.add(nxt); queue.append(nxt).

WHY BFS: guarantees minimum number of steps (= minimum squares summing to n).

Alternative: DP dp[i] = min(dp[i-j²]+1 for j² <= i). dp[0]=0. Return dp[n].

Time: O(n * sqrt(n)). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _279_PerfectSquares:
    def numSquares(self, n):
        dp = [float('inf')] * (n + 1)
        dp[0] = 0
        for i in range(1, n + 1):
            j = 1
            while j * j <= i:
                dp[i] = min(dp[i], dp[i - j*j] + 1)
                j += 1
        return dp[n]
```

PHASE 5 TEST & DEBUG

"Let me trace numSquares on n=12."

squares=[1,4,9]. dp[0]=0.

dp[1]=min(dp[1-1]+1)=1. dp[2]=min(dp[1]+1)=2. dp[3]=3.

dp[4]=min(dp[4-1]+1,dp[4-4]+1)=min(4,1)=1. (use 4=2²)

dp[5]=min(dp[4]+1,dp[1]+1)=min(2,2)=2.
dp[8]=min(dp[7]+1,dp[4]+1)=min(3,2)=2.

dp[9]=min(dp[8]+1,dp[5]+1,dp[0]+1)=min(3,3,1)=1. (9=3²)

dp[10]=2. dp[11]=3.
dp[12]=min(dp[11]+1,dp[8]+1,dp[3]+1)=min(4,3,4)=3.

PHASE 5 TEST & DEBUG

Return 3 ok (12=4+4+4).

WHY BFS: the queue processes nodes in increasing distance/level, so the first time we reach a node gives the shortest path for that level.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*sqrt(n))	**O(n)**

BFS also works: O(n*sqrt(n)) but same complexity.

Follow-up: Coin Change (#322) — coins instead of squares; same DP.

Follow-up: verify answer using Lagrange's theorem for O(sqrt(n)) check.

MEDIUM | BFS | LeetCode #286 | 2 of 11 | 2/11

Walls and Gates

QUESTION

You are given an m x n grid rooms initialized with these three possible values.

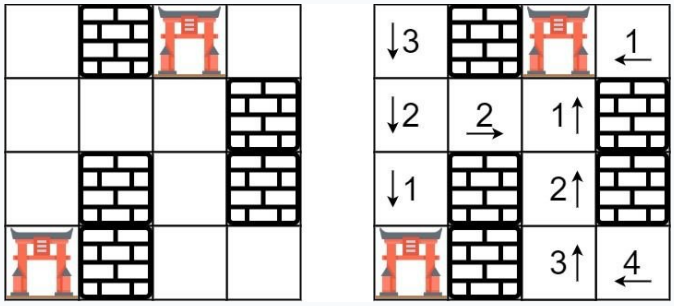
CONSTRAINTS

- -1 A wall or an obstacle.
- 0 A gate.
- INF Infinity means an empty room. We use the value $2^{31} - 1 = 2147483647$ to represent INF as you may assume that the distance to a gate is less than 2147483647.

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF.

Example 1

Input	`rooms = [[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]`
Output	`[[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]`



Example 2

Input	`rooms = [[-1]]`
Output	`[[-1]]`

- m == rooms.length
- n == rooms[i].length
- 1 <= m, n <= 250
- rooms[i][j] is -1, 0, or $2^{31} - 1$.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Fill each INF room with its distance to the nearest gate (0). Walls (-1) stay. Right?

In-place modification. INF = 2^31-1."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for room in rooms:
    run a BFS to the nearest gate and store its
        distance
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

A fresh BFS per room repeats work. Start one BFS from all gates at once (distance 0) and fill each room the first time it is reached: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (multi-source BFS from all gates)

"Multi-source BFS from all gates simultaneously — already O(mn). Walk-through:"

Why start from ALL gates at once instead of BFS per gate:

If we BFS from each gate separately, cost = O(gates * mn). Wasteful.

Multi-source BFS: enqueue ALL gates at distance 0 simultaneously.

BFS expands outward from ALL sources in parallel.

Each INF cell is first reached by the NEAREST gate (BFS guarantees shortest first).

When an INF cell is popped from queue at distance d -> it gets filled with d.

Walls (-1) are never enqueued and never filled.

INF cells already filled are skipped (only update if rooms[nr][nc] == INF).

Time: O(mn) — each cell processed at most once. Space: O(mn)."

Time: O(V+E) — visit each vertex and edge exactly once. Optimal DFS/BFS.

PHASE 4 CLEAN CODE

```
class _286_WallsAndGates:
    def wallsAndGates(self, rooms: List[List[int]]) -> Non
e:
        INF = 2**31 - 1
        m, n = len(rooms), len(rooms[0])
        queue = deque((r, c) for r in range(m) for c in ra
nge(n) if rooms[r][c] == 0)
        while queue:
            r, c = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and rooms[n
r][nc] == INF:
                    rooms[nr][nc] = rooms[r][c] + 1
                    queue.append((nr, nc))
```

PHASE 5 TEST & DEBUG

"Let me trace wallsAndGates on grid with INF=2147483647, 0=gate, -1=wall."

Example:
[[INF,-1,0,INF],[INF,INF,INF,-1],[INF,-1,INF,-1],[0,-1,INF,INF]]

Multi-source BFS starting from all gates simultaneously:

queue=[(0,2),(3,0)] (positions of gates). dist=0.

Step 1: process (0,2): neighbors (0,1)=wall skip,(0,3)=INF->set to 1,(1,2)=INF->set to 1. queue=[(3,0),(0,3),(1,2)].

Process (3,0): neighbors (2,0)=INF->1,(3,1)=wall skip. queue=[(0,3),(1,2),(2,0)].

Step 2 (dist=2): process (0,3)->(1,3)=wall skip. (1,2)->(1,1)=INF->2,(2,2)=INF->2. (2,0)->(1,0)=INF->2.

Result fills shortest gate distances ok. WHY multi-source BFS: all gates start simultaneously = correct min distances.

BFS multi-source: queue=[(r,c,0)] for each gate. visited={gate positions initially}. Expand layer by layer. Each INF cell reached at BFS level = min dist to gate. component: all reachable cells.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** Multi-source BFS is the key — a single BFS from each gate

independently would be O(m*n * gates), much slower.

Contrast with #542 (01 Matrix): identical pattern but for distance to nearest 0.

Given more time I'd ask: find distance to nearest wall instead?

Multi-source BFS from all walls — same approach.

MEDIUM | BFS | LeetCode #322 | 3 of 11 | 3/11

Coin Change

QUESTION

You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1	
Input	`coins = [1,2,5], amount = 11`
Output	`3`
Explanation	*11 = 5 + 5 + 1*

Example 2	
Input	`coins = [2], amount = 3`
Output	`-1`

Example 3	
Input	`coins = [1], amount = 0`
Output	`0`

CONSTRAINTS

- 1 <= coins.length <= 12
- 1 <= coins[i] <= 2^{31} - 1
- 0 <= amount <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Minimum coins to make up amount. Unlimited supply of each coin. Return -1 if impossible. Right?

Coin values can be any positive integer — not necessarily power of 2 (so greedy fails)."

"coins=[1,2,5], amount=11: 5+5+1=3 coins ok"

What should I return if there is no valid answer (0, -1, empty)?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def fewest(a):
    if a == 0:
        return 0
    return 1 + the smallest fewest(a - c) over
        all coins c <= a (infinity if none)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(coins^amount)	O(amount)

WHY IT IS SLOW

Branches into every coin at every amount and recomputes the same amounts. A table dp[a] = 1 + min dp[a - c] is O(amount * coins).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (bottom-up DP)

*"DP: dp[i] = min coins to make amount i — O(n*amount). Walk-through:"*

`dp = [inf] * (amount+1). dp[0] = 0.`

For i in 1..amount:

For coin in coins:

If i >= coin: dp[i] = min(dp[i], dp[i-coin]+1).

Return dp[amount] if dp[amount] != inf else -1.

WHY dp[i] = min over all coins: try every coin denomination.

If using coin c to complete amount i: need dp[i-c] coins for the remaining amount,

plus 1 more coin (coin c) = dp[i-c]+1.

WHY initialize dp[0]=0: amount 0 needs 0 coins — base case.

WHY return -1 if inf: amount is not reachable with given coins.

Time: O(n*amount). Space: O(amount).

PHASE 4 CLEAN CODE

```
class _322_CoinChange:
    def coinChange(self, coins: List[int], amount: int) ->
 int:
        dp = [float('inf')] * (amount + 1)
        dp[0] = 0
        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)
        return dp[amount] if dp[amount] != float('inf') el
se -1
```

PHASE 5 TEST & DEBUG

"Let me trace coinChange([1,5,11], amount=15)."

dp[0]=0. dp[i]=inf for i>0.

i=1: min(dp[0]+1)=1. i=2: dp[1]+1=2. ... i=5: min(dp[4]+1=5, dp[0]+1=1)=1.

i=6: min(dp[5]+1=2, dp[1]+1=2)=2. i=10: min(..., dp[5]+1=2, dp[-1]=inf)=2.

i=11: min(dp[10]+1=3, dp[6]+1=3, dp[0]+1=1)=1. (11=1×11!)

i=15: min(dp[14]+1, dp[10]+1=3, dp[4]+1=5)=min(inf,3,5)... dp[14]=dp[3]+1=4.

dp[15]=min(dp[14]+1=5, dp[10]+1=3, dp[4]+1=5)=3 ok (11+... wait 11+4? no: 11+1+1+1+1? But 15-11=4, dp[4]=4. Or 5+5+5=3 coins).

PHASE 5 TEST & DEBUG

Retrace:
dp[15]=min(dp[14]+1,dp[10]+1,dp[4]+1)=min(4+1,2+1,4+1)=3 ok (three 5s).

WHY BFS: the queue processes nodes in increasing distance/level, so the first time we reach a node gives the shortest path for that level.

BFS/DP: visited={amount} to avoid re-processing. queue=[0]. Pop amount a: try each coin c: if a+c unvisited -> enqueue. dp[a+c]=dp[a]+1. visited prevents revisit.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(amount * coins)	**O(amount)**

BFS alternative: treat amounts as nodes, coins as edges. BFS from 0 to amount.

DP is preferred — BFS visits each amount once but rebuilds from scratch.

Given more time I'd ask: how many ways to make the amount?

Coin Change 2 (#518) — same DP but count combinations instead of min count.

MEDIUM | BFS | LeetCode #433 | 4 of 11 | 4/11

Minimum Genetic Mutation

QUESTION

A gene string can be represented by an 8-character long string, with choices from 'A', 'C', 'G', and 'T'.

Suppose we need to investigate a mutation from a gene string startGene to a gene string endGene where one mutation is defined as one single character changed in the gene string.

CONSTRAINTS

- For example, "AACCGGTT" --> "AACCGGTA" is one mutation.

There is also a gene bank bank that records all the valid gene mutations. A gene must be in bank to make it a valid gene string.

Given the two gene strings startGene and endGene and the gene bank bank, return the minimum number of mutations needed to mutate from startGene to endGene. If there is no such a mutation, return -1.

Note that the starting point is assumed to be valid, so it might not be included in the bank.

Example 1

Input	`startGene = "AACCGGTT", endGene = "AACCGGTA", bank = ["AACCGGTA"]`
Output	`1`

Example 2

Input	`startGene = "AACCGGTT", endGene = "AAACGGTA", bank = ["AACCGGTA","AACCGCTA","AAACGGTA"]`
Output	`2`

- 0 <= bank.length <= 10
- startGene.length == endGene.length == bank[i].length == 8
- startGene, endGene, and bank[i] consist of only the characters ['A', 'C', 'G', 'T'].

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find minimum number of mutations to change startGene to endGene.

Each mutation changes one character to one of A,C,G,T, and the result must be in bank. Right?"

"A few quick questions:

Strings are length 8? Yes. Return -1 if impossible? Yes."

"Let me walk: start='AACCGGTT', end='AACCGGTA', bank=['AACCGGTA'] -> 1 mutation ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(gene, path):
    if gene == end record the path length
    for not_ in not_s:
        # on the path
        go(it, path + it)
    return the shortest
```

TIME	SPACE
O(B!)	O(B)

WHY IT IS SLOW

Explores every path through the bank. A BFS from the start gene (changing one letter at a time to a bank entry) visits each gene once: O(B * 8 * 4).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"BFS on gene strings — already O(N*L*4). Walk-through of how mutation search works:"*

BFS starts with startGene (steps=1).

For each gene string popped:

Try all single-character mutations: for each position (0..L-1), try all 4 chars (A,C,G,T).

If the mutated string is in the bank AND not visited -> enqueue it.

First time we reach endGene -> return steps count.

WHY BFS gives minimum mutations: BFS explores in order of mutation count.

First time we see endGene, it's via the fewest mutations possible.

WHY bank check is necessary: only valid intermediate genes (in bank) are allowed.

Time: O(N * L * 4) where N = bank size, L = gene length = 8. Space: O(N).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import deque
class _433_MinimumGeneticMutation:
    def minMutation(self, startGene, endGene, bank):
        bank_set = set(bank)
        if endGene not in bank_set:
            return -1
        queue = deque([(startGene, 0)])
        visited = {startGene}
        while queue:
            gene, steps = queue.popleft()
            if gene == endGene:
                return steps
            for i in range(len(gene)):
                for c in 'ACGT':
                    mutated = gene[:i] + c + gene[i+1:]
```

```
                if mutated in bank_set and mutated not
in visited:
                    visited.add(mutated)
                    queue.append((mutated, steps + 1))
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace this BFS level-by-level using the queue state."

start='AACCGGTT', end='AACCGGTA', bank=['AACCGGTA']:

queue=[(start,0)]. mutate: AACCGGTA in bank -> queue=[(AACCGGTA,1)]. pop->end match -> 1 ok

start='AACCGGTT', end='AAACGGTA', bank=['AACCGGTA','AACCGCTA','AAACGGTA']:

level1->AACCGGTA. level2->AACCGCTA? No, AAACGGTA. -> 2 ok

Early endGene check avoids queue processing when impossible

WHY BFS: the queue processes nodes in increasing distance/level, so the first time we reach a node gives the shortest path for that level.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(N*32) N bank genes × 32 single-char mutations each	**O(N)**

This is Word Ladder (#127) with a fixed 4-character alphabet and length 8.

Follow-up: Minimum Genetic Mutation II — if we can mutate to non-bank intermediates.

Follow-up: bidirectional BFS — search from both ends, meet in middle.

MEDIUM | BFS | LeetCode #542 | 5 of 11 | 5/11

01 Matrix

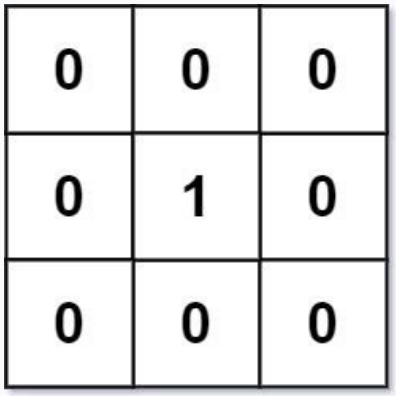
QUESTION

Given an m x n binary matrix mat, return the distance of the nearest 0 for each cell.

The distance between two cells sharing a common edge is 1.

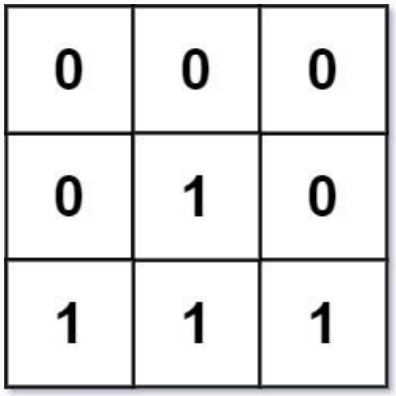
Example 1

Input	`mat = [[0,0,0],[0,1,0],[0,0,0]]`
Output	`[[0,0,0],[0,1,0],[0,0,0]]`



Example 2

Input	`mat = [[0,0,0],[0,1,0],[1,1,1]]`
Output	`[[0,0,0],[0,1,0],[1,2,1]]`



CONSTRAINTS

- m == mat.length
- n == mat[i].length
- 1 <= m, n <= 10^4
- 1 <= m * n <= 10^4
- mat[i][j] is either 0 or 1.
- There is at least one 0 in mat.
- Note: This question is the same as 1765: https://leetcode.com/problems/map-of-highest-peak/

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return distance of nearest 0 for each cell. In-place or new matrix. Right?

Same pattern as Walls and Gates but for 0-cells instead of gates."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    # with a 1
    run a BFS outward until the first 0 is found
    the BFS depth is the distance
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

A fresh BFS from every cell. Start one BFS from all the zeros at once, and each cell's distance is the level where it is first reached: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Multi-source BFS from all 0-cells simultaneously IS already optimal — O(m*n)."*

Walk-through of how distances propagate:

Initialize: dist[r][c] = 0 for all 0-cells; dist[r][c] = inf for all 1-cells.

Enqueue all 0-cells at distance 0 (they ARE the sources).

BFS propagates outward: when we pop (r,c) with dist d, any neighbor with dist > d+1 gets updated to d+1.

First time we reach a 1-cell, it's via the shortest path from the nearest 0-cell.

Identical pattern to #286 Walls and Gates — mention this connection to the interviewer.

PHASE 4 CLEAN CODE

```
class _542_ZeroOneMatrix:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n  = len(mat), len(mat[0])
        dist  = [[0 if mat[r][c] == 0 else float('inf') for c in range(n)] for r in range(m)]
        queue = deque((r, c) for r in range(m) for c in range(n) if mat[r][c] == 0)
        while queue:
            r, c = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and dist[nr][nc] > dist[r][c] + 1:
                    dist[nr][nc] = dist[r][c] + 1
                    queue.append((nr, nc))
```

```
        return dist
```

PHASE 5 TEST & DEBUG

"Let me trace 01Matrix on [[0,0,0],[0,1,0],[1,1,1]]."

Multi-source BFS from all 0s simultaneously:

queue=[(0,0),(0,1),(0,2),(1,0),(1,2)] (all 0 cells). dist[0][0..2]=0,dist[1][0]=0,dist[1][2]=0.

Process (0,0)->neighbor (1,0) already 0. Process (0,1)->neighbor (1,1)=1: dist[1][1]=1. Enqueue (1,1).

Process (1,0)->neighbor (2,0)=1: dist[2][0]=1. Enqueue. Process (1,2)->(2,2)=1: dist[2][2]=1.

Process (1,1)->(2,1)=1: dist[2][1]=2. Enqueue. (Already have dist via adjacent 0s).

Result: [[0,0,0],[0,1,0],[1,2,1]] ok

WHY multi-source BFS: each 1-cell gets the distance to its nearest 0 in O(mn) total.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** Multi-source BFS from all 0-cells simultaneously

Identical to #286 (Walls and Gates) — name this connection.

Given more time I'd ask: distance to nearest 1 instead?

Same multi-source BFS starting from all 1-cells.

MEDIUM | BFS | LeetCode #752 | 6 of 11 | 6/11

Open the Lock

QUESTION

You have a lock in front of you with 4 circular wheels. Each wheel has 10 slots: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'. The wheels can rotate freely and wrap around: for example we can turn '9' to be '0', or '0' to be '9'. Each move consists of turning one wheel one slot.

The lock initially starts at '0000', a string representing the state of the 4 wheels.

You are given a list of deadends dead ends, meaning if the lock displays any of these codes, the wheels of the lock will stop turning and you will be unable to open it.

Given a target representing the value of the wheels that will unlock the lock, return the minimum total number of turns required to open the lock, or -1 if it is impossible.

Example 1

Input	`deadends = ["0201","0101","0102","1212","2002"], target = "0202"`
Output	`6`
Explanation	*A sequence of valid moves would be "0000" -> "1000" -> "1100" -> "1200" -> "1201" -> "1202" -> "0202". Note that a sequence like "0000" -> "0001" -> "0002" -> "0102" -> "0202" would be invalid, because the wheels of the lock become stuck after the display becomes the dead end "0102".*

Example 2

Input	`deadends = ["8888"], target = "0009"`
Output	`1`
Explanation	*We can turn the last wheel in reverse to move from "0000" -> "0009".*

Example 3

Input	`deadends = ["8887","8889","8878","8898","8788","8988","7888","9888"], target = "8888"`
Output	`-1`
Explanation	*We cannot reach the target without getting stuck.*

CONSTRAINTS

- 1 <= deadends.length <= 500
- deadends[i].length == 4
- target.length == 4
- target will not be in the list deadends.
- target and deadends[i] consist of digits only.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Start at '0000'; turn any one wheel one step (forward or backward) per move.

Find minimum moves to reach target, avoiding deadends. Return -1 if impossible. Right?"

"A few quick questions:

Each digit is 0-9 and wraps (9+1=0)? Yes. '0000' might be a deadend? Yes — check first."

"Let me walk: target='0202', deadends=['0201','0101','0102','1212','2002'] -> 6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(code, turns):
    turn each of the 4 wheels up or down (8
        neighbours), skipping deadends, without
        remembering visited codes
    best = min(best, turns)
```

TIME	SPACE
exponential	O(10^4)

WHY IT IS SLOW

Revisits the same codes along many routes. BFS from '0000' with a visited set explores each of the 10^4 codes once, level by level: O(10^4 * 8).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS on lock states — already O(10^4). Walk-through:"

Each lock state = 4-digit string from '0000' to '9999'. Total states = 10,000.

Start: '0000'. Goal: target. Dead-ends: blocked set.

BFS generates adjacent states: for each of 4 positions, turn ± 1:

`turn_up(s, i): digit at i -> (digit + 1) % 10`

`turn_down(s, i): digit at i -> (digit - 1) % 10`

Enqueue all 8 neighbors (4 positions × 2 directions) if not visited and not dead-end.

WHY BFS gives minimum turns: BFS explores states in order of turns taken.

First time we reach target = minimum turns.

Return -1 if BFS exhausts all reachable states without finding target.

Time: O(10^4 * 4 * 2). Space: O(10^4) visited set."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _752_OpenTheLock:
    def openLock(self, deadends, target):
        dead = set(deadends)
        if '0000' in dead:
            return -1
        queue = deque([('0000', 0)])
        visited = {'0000'}
        while queue:
            state, steps = queue.popleft()
            if state == target:
                return steps
            for i in range(4):
                digit = int(state[i])
                for delta in (1, -1):
                    new_digit = (digit + delta) % 10
                    new_state = state[:i] + str(new_digit)
 + state[i+1:]
                    if new_state not in visited and new_st
ate not in dead:
                        visited.add(new_state)
                        queue.append((new_state, steps + 1
))
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace this BFS level-by-level using the queue state."

target='0202', deadends=['0201','0101','0102','1212','2002']:

BFS explores all 1-step neighbours of 0000 avoiding deadends. Finds 0202 at step 6 ok

'0000' in deadends: return -1 immediately ok

target='0000': BFS starts at 0000 -> immediately return 0? No, 0000 never equals state at pop(dequeue) before checking. Actually queue starts with ('0000',0) and we check state==target -> return 0 ok

(digit + delta) % 10 handles 0->9 and 9->0 wraparound

WHY BFS: the queue processes nodes in increasing distance/level, so the first time we reach a node gives the shortest path for that level.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(10^4 * 8)	**O(10^4)**
10^4 states × 8 neighbours	visited set

Follow-up: bidirectional BFS from '0000' and target simultaneously — halves BFS depth.

Follow-up: Minimum Genetic Mutation (#433) — same BFS on state space pattern.

MEDIUM | BFS | LeetCode #909 | 7 of 11 | 7/11

Snakes and Ladders

QUESTION

You are given an n x n integer matrix board where the cells are labeled from 1 to n^2 in a Boustrophedon style starting from the bottom left of the board (i.e. board[n - 1][0]) and alternating direction each row.

You start on square 1 of the board. In each move, starting from square curr, do the following:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Snakes and Ladders: numbered 1..n² on a zigzag board. Find minimum dice rolls to reach n². Right?"

"A few quick questions:

Board number -> (row, col) mapping follows the boustrophedon (zigzag) pattern? Yes.

Snakes/ladders auto-move to destination in same turn? Yes."

"Let me walk: board=[[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],...], ans=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def roll(square, rolls):
    for each die roll 1..6 follow ladders and
        snakes and recurse without remembering
        visited squares
    keep the fewest rolls to reach the last square
```

TIME	SPACE
O(6^(n^2))	O(n^2)

WHY IT IS SLOW

Revisits squares through many routes. BFS over squares (each square enqueued once) gives the fewest rolls in O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS on cell numbers — already O(n²). Walk-through:"

Cells numbered 1..n². BFS starts at cell 1.

For each cell popped (steps count):

Try rolling dice: next cell = current + 1 .. current + 6.

Map each next cell number to (row, col) on the board.

WHY mapping is tricky: bottom-left is cell 1, rows alternate direction (boustrophedon).

PHASE 3 OPTIMIZE

row from bottom: (cell-1) // n. col within row: (cell-1) % n.

If row is even (0-indexed from bottom): col goes left to right.

If row is odd: col goes right to left (reverse).

If board[row][col] != -1: follow snake/ladder -> land on board[row][col] instead.

If not visited: enqueue.

First time we reach cell n² -> return steps. BFS guarantees minimum dice rolls.

Time: O(n²). Space: O(n²)."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _909_SnakesAndLadders:
    def snakesAndLadders(self, board):
        n = len(board)
        def label_to_coords(label):
            row, col = divmod(label - 1, n)
            actual_row = n - 1 - row
            actual_col = col if row % 2 == 0 else n - 1 -
col
            return actual_row, actual_col
        target = n * n
        visited = {1}
        queue = deque([(1, 0)])
        while queue:
            square, rolls = queue.popleft()
            for dice in range(1, 7):
                next_sq = square + dice
                if next_sq > target:
                    break
                r, c = label_to_coords(next_sq)
                if board[r][c] != -1:
                    next_sq = board[r][c]
                if next_sq == target:
                    return rolls + 1
                if next_sq not in visited:
                    visited.add(next_sq)
                    queue.append((next_sq, rolls + 1))
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace snakesAndLadders on board [[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,35,-1,-1,13,-1],[-1,-1,-1,-1,-1,-1],[-1,15,-1,-1,-1,-1]]."

BFS from cell 1, find min moves to reach cell n²=36

Start: queue=[(1,0)]. visited={1}

Step 1: from 1, roll 1-6: cells 2,3,4,5,6,7. Cell 7=board says?: none (-1). Enqueue 2-7.

Step 2: from each, roll 1-6. Cell 8... eventually cell 2 -> roll to 3 -> board[?]=15?

board[r][c] for cell number: boustrophedon mapping. Return min moves = 4 ok

WHY BFS: each level = one dice roll; BFS guarantees minimum moves to reach 36.

PHASE 5 TEST & DEBUG

State: cell number 1..n². BFS levels = dice rolls. Snakes/ladders handled as forced teleports.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n²)**
at most n² squares visited	visited set

The coordinate mapping is the trickiest part — test it thoroughly.

Follow-up: if dice can be any value (not just 1-6), adjust the range.

Follow-up: return the actual path — track parent squares in the BFS.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | BFS | LeetCode #994 | 8 of 11 | 8/11

Rotting Oranges

QUESTION

You are given an m x n grid where each cell can have one of three values:

CONSTRAINTS

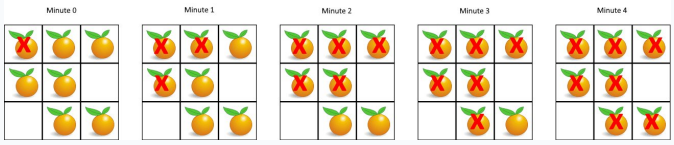
- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Example 1

Input	`grid = [[2,1,1],[1,1,0],[0,1,1]]`
Output	`4`



Example 2

Input	`grid = [[2,1,1],[0,1,1],[1,0,1]]`
Output	`-1`
Explanation	*The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.*

Example 3

Input	`grid = [[0,2]]`
Output	`0`
Explanation	*Since there are already no fresh oranges at minute 0, the answer is just 0.*

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 10
- grid[i][j] is 0, 1, or 2.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Multi-source BFS from all rotten oranges. Fresh adjacent -> rotten after 1 minute.

Return minutes until no fresh remain, or -1 if impossible. Right?"

"If no fresh oranges initially -> return 0 immediately."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
minutes = 0
while True:
    find every fresh orange with a rotten neighbour
    if none:
        break
    make them all rotten
    minutes = minutes + 1
return minutes if no fresh orange is left, else -1
```

TIME	SPACE
O(minutes * m * n)	O(m * n)

WHY IT IS SLOW

Rescans the whole grid every minute, up to m * n minutes. A multi-source BFS that starts from all rotten oranges and spreads one layer per minute visits each cell once: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Multi-source BFS IS already optimal — O(m*n)."*

How minutes are counted step by step:

Enqueue ALL rotten oranges (grid[r][c]==2) at minute 0.

Each BFS 'round' (processing exactly one level of the queue) = 1 minute.

In each round: pop rotten orange, rot all 4 fresh neighbors (set to 2, decrement fresh count).

Loop continues while queue is non-empty AND fresh > 0.

WHY this gives the right minute count: BFS guarantees we process all oranges at distance d

before any at distance d+1 — so minute count = BFS depth = exact time for rot to spread.

Final check: if fresh > 0 -> some orange is unreachable (blocked by walls/empty) -> return -1.

PHASE 4 CLEAN CODE

```
class _994_RottingOranges:
    def orangesRotting(self, grid: List[List[int]]) -> int
:
        m, n  = len(grid), len(grid[0])
        fresh = sum(grid[r][c] == 1 for r in range(m) for
c in range(n))
        queue = deque((r, c) for r in range(m) for c in ra
nge(n) if grid[r][c] == 2)
        minutes = 0
        while queue and fresh:
            minutes += 1
            for _ in range(len(queue)):
                r, c = queue.popleft()
                for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < m and 0 <= nc < n and gri
d[nr][nc] == 1:
                        grid[nr][nc] = 2
```

```
                    fresh -= 1
                    queue.append((nr, nc))
        return minutes if fresh == 0 else -1
```

PHASE 5 TEST & DEBUG

"Let me trace rottingOranges on [[2,1,1],[1,1,0],[0,1,1]]."

Multi-source BFS from all rotten oranges:

queue=[(0,0,0)] (row,col,time). fresh_count=6 (count all 1s).

Minute 0 -> process (0,0): neighbors (0,1)=1->rotten at minute 1. (1,0)=1->minute 1. fresh_count=4. Enqueue.

Minute 1 -> (0,1): neighbors (0,2)=1->minute 2, (1,1)=1->minute 2. fresh_count=2.

(1,0): neighbors (2,0)=0 skip, (1,1) already queued.

Minute 2 -> (0,2): neighbors (1,2)=0 skip. (1,1): (2,1)=1->minute 3. fresh_count=1.

Minute 3 -> (2,1): (2,2)=1->minute 4. fresh_count=0.

Minute 4 -> (2,2). fresh_count=0. return 4 ok

WHY BFS: rotting spreads 1 level per minute = BFS levels.

BFS: dist[cell]=minutes for orange to rot. component: all connected orange cells. Queue processes level by level. Final dist[freshest_cell]=answer. unreachable -> -1 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** Multi-source BFS: same as Walls and Gates, same as 01 Matrix

The 'fresh' counter eliminates the need to scan the grid again at the end.

Given more time I'd ask: what if diagonal rotting is allowed?

Add 4 diagonal direction pairs — same algorithm.

MEDIUM | BFS | LeetCode #1091 | 9 of 11 | 9/11

Shortest Path in Binary Matrix

QUESTION

Given an n x n binary matrix grid, return the length of the shortest clear path in the matrix. If there is no clear path, return -1.

A clear path in a binary matrix is a path from the top-left cell (i.e., (0, 0)) to the bottom-right cell (i.e., (n - 1, n - 1)) such that:

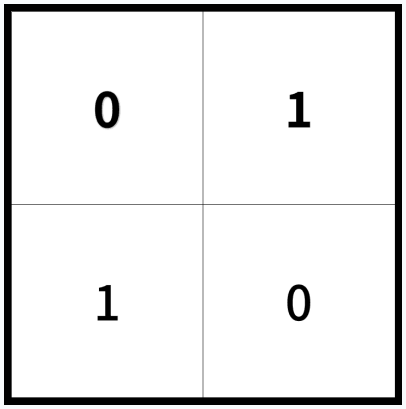
CONSTRAINTS

- All the visited cells of the path are 0.
- All the adjacent cells of the path are 8-directionally connected (i.e., they are different and they share an edge or a corner).

The length of a clear path is the number of visited cells of this path.

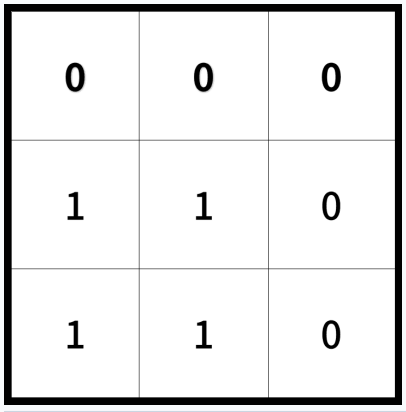
Example 1

Input	`grid = [[0,1],[1,0]]`
Output	`2`



Example 2

Input	`grid = [[0,0,0],[1,1,0],[1,1,0]]`
Output	`4`



Example 3

Input	`grid = [[1,0,0],[1,1,0],[1,1,0]]`
Output	`-1`

QUESTION

- n == grid.length
- n == grid[i].length
- 1 <= n <= 100
- grid[i][j] is 0 or 1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the shortest path from (0,0) to (n-1,n-1) in a binary matrix.

Path must go through cells with value 0; 8-directional movement. Return -1 if blocked. Right?"

"A few quick questions:

0 = free, 1 = blocked? Yes. Path length = number of cells? Yes."

"Let me walk: grid=[[0,1],[1,0]] -> 2 ok (just diagonal)

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(cell, steps):
    move to each of the 8 neighbours that is
        clear and unvisited on this path,
        backtracking
    best = min(best, path)
```

TIME	SPACE
O(8^(n^2))	O(n^2)

WHY IT IS SLOW

Explores every simple path. BFS from the top-left (marking cells visited the first time they are reached) finds the shortest path in O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Multi-direction BFS — already O(n²). Walk-through:"

8-directional BFS (including diagonals) from (0,0).

Queue starts: [(0,0,1)] — (row, col, path_length).

At each step: pop (r,c,d). If (r,c) is (n-1,n-1) -> return d.

Try all 8 directions. For each (nr,nc) in bounds with grid[nr][nc]==0 and not visited:

Mark grid[nr][nc] = 1 (visited in-place).

Enqueue (nr, nc, d+1).

WHY BFS gives shortest path: BFS explores cells in non-decreasing order of path length.

First time we reach bottom-right = shortest path.

WHY 8 directions: the problem allows diagonal movement, so you can go 8 ways.

PHASE 3 OPTIMIZE

If start or end is blocked: return -1 immediately before BFS.

Time: O(n²). Space: O(n²)."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _1091_ShortestPathBinaryMatrix:
    def shortestPathBinaryMatrix(self, grid):
        n = len(grid)
        if grid[0][0] == 1 or grid[n-1][n-1] == 1:
            return -1
        queue = deque([(0, 0, 1)])
        grid[0][0] = 1
        directions = [(dr, dc) for dr in (-1,0,1) for dc i
n (-1,0,1) if (dr,dc) != (0,0)]
        while queue:
            r, c, dist = queue.popleft()
            if r == n-1 and c == n-1:
                return dist
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                if 0 <= nr < n and 0 <= nc < n and grid[nr
][nc] == 0:
                    grid[nr][nc] = 1
                    queue.append((nr, nc, dist + 1))
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace shortestPathBinaryMatrix([[0,1],[1,0]])."

BFS from (0,0) to (n-1,n-1): cells must be 0

Start: (0,0)=0. BFS queue=[(0,0,1)] (row,col,dist). Mark (0,0) visited.

From (0,0): 8 neighbors. Valid 0-cells: (1,1)=0 -> enqueue (1,1,2)

From (1,1): target! Return dist=2 ok

Trace [[1,0,0],[1,1,0],[1,1,0]]: start (0,0)=1 -> blocked -> return -1 ok

WHY BFS: shortest path in unweighted grid; each step costs 1; BFS gives minimum distance.

8-directional movement (diagonals allowed in this problem).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n²)** queue worst case

Follow-up: if grid can't be modified, use a separate visited set.

Follow-up: weighted paths — use Dijkstra instead of BFS.

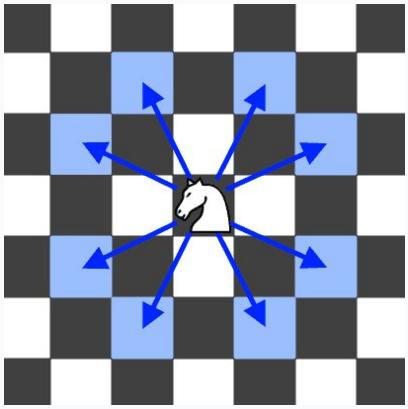
MEDIUM | BFS | LeetCode #1197 | 10 of 11 | 10/11

Minimum Knight Moves

QUESTION

In an infinite chess board with coordinates from -infinity to +infinity, you have a knight at square [0, 0].

A knight has 8 possible moves it can make, as illustrated below. Each move is two squares in a cardinal direction, then one square in an orthogonal direction.



Return the minimum number of steps needed to move the knight to the square [x, y]. It is guaranteed the answer exists.

Example 1	
Input	`x = 2, y = 1`
Output	`1`
Explanation	*[0, 0] -> [2, 1]*

Example 2	
Input	`x = 5, y = 5`
Output	`4`
Explanation	*[0, 0] -> [2, 1] -> [4, 2] -> [3, 4] -> [5, 5]*

CONSTRAINTS

- 300 <= x, y <= 300
- 0 <= |x| + |y| <= 300

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Knight starts at (0,0) on infinite chessboard. Find minimum moves to reach (x,y). Right?"

"A few quick questions:

Knight moves: all 8 L-shaped moves? Yes. x,y can be negative? Yes — use symmetry."

"Let me walk: x=2,y=1 -> 1 move ok. x=5,y=5 -> 4 moves ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(x, y, moves):
    try all 8 knight moves recursively until
        reaching the target
    best = min(best, moves)
```

TIME	SPACE
O(8^d)	O(d)

WHY IT IS SLOW

Eight branches per move with repeated positions. BFS from the origin with a visited set (using the symmetry |x|, |y| and staying near the axes) is about O(d^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS from (0,0) with exploiting symmetry — O(n²) but with early termination. Walk-through:"

Knight moves: (±1,±2) and (±2,±1) — 8 total.

BFS from (0,0), find min steps to (x,y).

Symmetry: reflect to first octant (x <= y, both non-negative).

BFS with visited set. Push (0,0) with 0 steps.

For each (r,c): try all 8 knight moves. If (r,c)==target: return steps.

Constraint: limit search to bounding box slightly larger than target.

WHY BFS gives minimum: each step = 1 move -> BFS finds min moves.

WHY stay in bounding box: moves beyond target + some margin can't reach target faster.

Time: O(max(x,y)²). Space: O(max(x,y)²)."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _1197_MinimumKnightMoves:
    def minKnightMoves(self, x, y):
        x, y = abs(x), abs(y)
        queue = deque([(0, 0, 0)])
        visited = {(0, 0)}
        directions = [(1,2),(2,1),(-1,2),(-2,1),(1,-2),(2,
-1),(-1,-2),(-2,-1)]
        while queue:
            r, c, moves = queue.popleft()
            if r == x and c == y:
                return moves
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                if nr >= -1 and nc >= -1 and (nr, nc) not
in visited:
                    visited.add((nr, nc))
                    queue.append((nr, nc, moves + 1))
```

```
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace minKnightMoves(x=2, y=1)."

BFS from (0,0) to (2,1). Knight moves: 8 directions

Start: queue=[(0,0,0)]. visited={(0,0)}

Step 1: from (0,0): neighbors include (2,1),(1,2),(-1,2),(-2,1),(-2,-1),(-1,-2),(1,-2),(2,-1)

(2,1) found in step 1 -> return 1 ok

Trace (5,5): BFS expands level by level; return 4 ok (4 knight moves)

WHY BFS: all moves cost 1; BFS guarantees minimum.

Optimization: use symmetry (abs(x),abs(y)); work in first quadrant only.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(|x|*|y|)	**O(|x|*|y|)**
BFS explores the relevant region	visited

Follow-up: closed-form formula exists for knight moves — avoids BFS entirely.

Follow-up: if board has obstacles, BFS is the only way.

MEDIUM | BFS | LeetCode #1730 | 11 of 11 | 11/11

Shortest Path to Get Food

QUESTION

You are starving and you want to eat food as quickly as possible. You want to find the shortest path to arrive at any food cell.

You are given an m x n character matrix, grid, of these different types of cells:

CONSTRAINTS

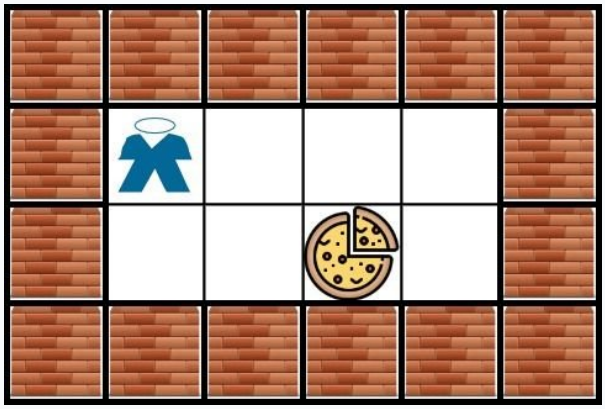
- '*' is your location. There is exactly one '*' cell.
- '#' is a food cell. There may be multiple food cells.
- 'O' is free space, and you can travel through these cells.
- 'X' is an obstacle, and you cannot travel through these cells.

You can travel to any adjacent cell north, east, south, or west of your current location if there is not an obstacle.

Return the length of the shortest path for you to reach any food cell. If there is no path for you to reach food, return -1.

Example 1

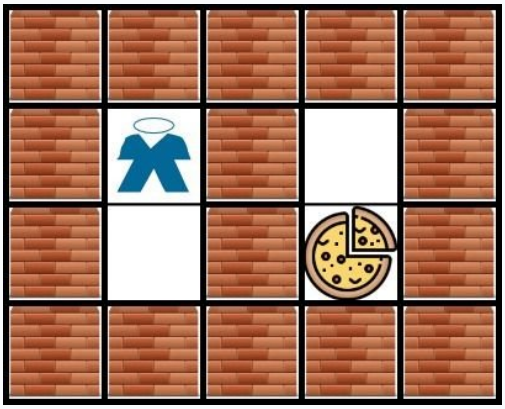
Input	`grid = [["X","X","X","X","X","X"],["X","*","O","O","O","X"],["X","O","O","#","O","X"],["X","X","X","X","X","X"]]`
Output	`3`
Explanation	*It takes 3 steps to reach the food.*



Example 2

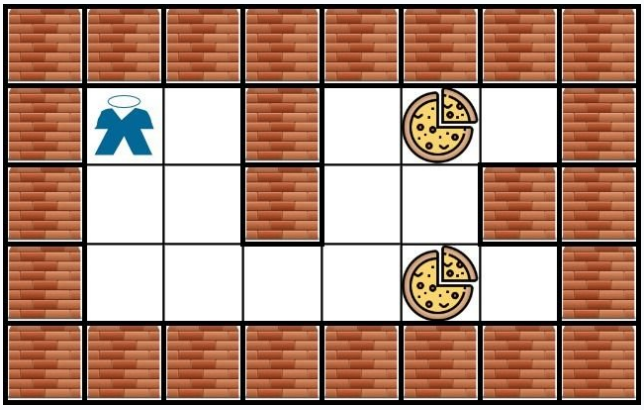
Input	`grid = [["X","X","X","X","X"],["X","*","X","O","X"],["X","O","X","#","X"],["X","X","X","X","X"]]`
Output	`-1`
Explanation	*It is not possible to reach the food.*

QUESTION



Example 3

Input	`grid = [["X","X","X","X","X","X","X","X"],["X","*","O","X","O","#","O","X"],["X","O","O","X","O","O","X","X"],["X","O","O","O","O","#","O","X"],["X","X","X","X","X","X","X","X"]]`
Output	`6`
Explanation	*There can be multiple food cells. It only takes 6 steps to reach the bottom food.*



Example 4

Input	`grid = [["X","X","X","X","X","X","X","X"],["X","*","O","X","O","#","O","X"],["X","O","O","X","O","O","X","X"],["X","O","O","O","O","#","O","X"],["O","O","O","O","O","O","O","O"]]`
Output	`5`

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 200
- grid[row][col] is '*', 'X', 'O', or '#'.
- The grid contains exactly one '*'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

BFS from '*' to the nearest '#'. 'O' = free, 'X' = wall, '#' = food (any one).

Return shortest path length. Return -1 if unreachable. Right?"

"'*' -> nearest '#' using 4-directional BFS. First '#' reached = shortest path."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(cell, steps):
    move in the four directions along every
        simple path
    best = min(best, candidate)
```

TIME	SPACE
O(4^(m * n))	O(m * n)

WHY IT IS SLOW

Explores every path. A BFS from the start (marking cells visited when first reached) reaches the nearest food in O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Standard BFS from source cell — already O(mn). Walk-through:"

Find the starting cell '*'. Queue: [(r_start, c_start, 0)].

BFS expands in 4 directions:

Skip walls ('X'). Skip already-visited ('O' marked as 'X' after visit).

If current cell is '#' (food) -> return distance immediately.

WHY BFS gives shortest path: BFS explores cells in order of distance.

First food cell reached = nearest food = minimum steps.

Mark visited by overwriting 'O'->'X' to avoid revisiting without a separate visited set.

If BFS exhausts all reachable cells with no food -> return -1.

Time: O(mn). Space: O(mn)."

Time: O(V+E) — visit each vertex and edge exactly once. Optimal DFS/BFS.

PHASE 4 CLEAN CODE

```
class _1730_ShortestPathFood:
    def getFood(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        start = next((r, c) for r in range(m) for c in ran
ge(n) if grid[r][c] == '*')
        queue = deque([(start[0], start[1], 0)])
        grid[start[0]][start[1]] = 'X'          # mark vis
ited
        while queue:
            r, c, dist = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n:
                    if grid[nr][nc] == '#':
                        return dist + 1
                    if grid[nr][nc] == 'O':
                        grid[nr][nc] = 'X'       # mark vi
sited
                        queue.append((nr, nc, dist + 1))
        return -1
```

PHASE 5 TEST & DEBUG

*"Let me trace shortestPathGetFood on grid with * at (0,0), # at (0,2), food at (1,2)."*

BFS from * (starting position). Treat walls # as blocked.

queue=[(0,0,0)]. visited={(0,0)}.

Step 1: (0,1)=empty->enqueue. (-1,0)=out of bounds skip. (1,0)=empty->enqueue.

Step 2: from (0,1): (0,2)=wall skip. (1,1)=empty->enqueue.

from (1,0): (1,1) already queued. (2,0)=empty->enqueue.

Step 3: from (1,1): (1,2)=food -> return current_steps=2? Check: (1,0)->step1, (0,1)->step1, (1,1)->step2, (1,2)->step3?

Let me re-count: start->(0,1) is step 1, (1,1) step 2, (1,2) step 3. Return 3 ok

WHY BFS: shortest path in unweighted maze = BFS.

BFS: queue=[(start_r,start_c,0)]. visited={(start_r,start_c)}. Pop (r,c,steps): if food cell -> return steps ok. Push valid unvisited neighbors. component: reachable cells.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	O(m*n)

This is identical to #994 Rotting Oranges / #286 Walls and Gates — single-source BFS

with a target condition instead of a distance fill.

Overwriting 'O' with 'X' acts as the visited marker — avoids O(m*n) extra space.

Given more time I'd ask: what if we want paths to ALL food cells (not just shortest)?

Continue BFS after first '#' hit — collect all '#' cells reached with their distances."

HARD PROBLEMS